

NEMIX Capstone 2 Project Audit Report

Consumable Supply and Inventory Management System with RFID Integration

Audit date: 15 September 2026 | Repository revision: d853b87eda07

Executive assessment

NEMIX has substantial implementation evidence for a Capstone 2 demonstration: inventory receiving and issuance, batch valuation, supplier management, report snapshots, access control, and audit trails. The project can support a structured demonstration after rehearsal. Production readiness is not established because the audit confirmed access-control weaknesses and identified unresolved verification and deployment gaps.

Verification	Observed result
Existing automated tests	272 passed / 273 executed; 1 failure
Assertions and runtime	1,682 assertions; 45.627 seconds; PHP 8.2.12
TypeScript check	Passed with exit code 0
Frontend build	Passed in 23.61 seconds
ESP32 firmware compile	Passed; 78% program storage, 14% global memory
Additional audit checks	3 characterization tests confirmed access weaknesses
Physical RFID and user acceptance	Not exercised in this audit

Main strengths

The inventory services use database transactions and row locks; FIFO allocates the oldest eligible receiving batches first. Tests cover stock changes, insufficient-stock rejection, reversal, valuation, historical report snapshots, signatories, and staff authorization. These are useful technical contributions to explain during the defense. [S2-S5]

Main concerns

RFID authentication accepts a spoofable device header and fails open when no token is configured. Verified users without permissions can open inventory and RFID pages. Firmware contains a configured Wi-Fi password in a tracked file and disables TLS certificate verification. Address these before external deployment. [S6-S9]

Presentation position

Describe NEMIX as an implemented inventory management prototype with automated verification and identified hardening work. The 99.63% test pass rate describes this test run only; it is not a completion percentage, security score, or user acceptance rating.

Project Context and Objectives

System purpose

The application consolidates consumable inventory records, receiving, issuance, supplier information, and reporting in a browser-based system. Its current branding identifies the University of Camarines Norte and its supply and inventory context. This describes the implementation; it does not establish institutional adoption. [S1]

Problem addressed

The system is designed to address fragmented stock records, difficulty tracing stock movements, repeated preparation of inventory forms, and limited visibility into quantities and values. These are the problem areas implied by the implemented workflows. Claims about the institution's actual baseline should be supported by interviews, observations, or approved project documentation.

General objective

Provide a centralized inventory application that records stock movement, maintains supplier and batch traceability, generates inventory reports, and supports RFID-assisted item identification.

Specific objectives and implementation evidence

Objective	Evidence in the project
Control stock movement	Receiving and issuance services; validation and reversal tests
Preserve cost history	Inventory batches and issuance allocations with recorded costs
Improve traceability	Login trails, transaction trails, ownership checks, report snapshots
Support report preparation	RSMI, RPCI, Stock Card, and Memorandum Receipt datasets
Manage authorized access	Roles, route permissions, staff invitations, verification, OTP password change
Identify tagged items	ESP32/YRM100 firmware and Laravel lookup/live-feed endpoints

Intended users

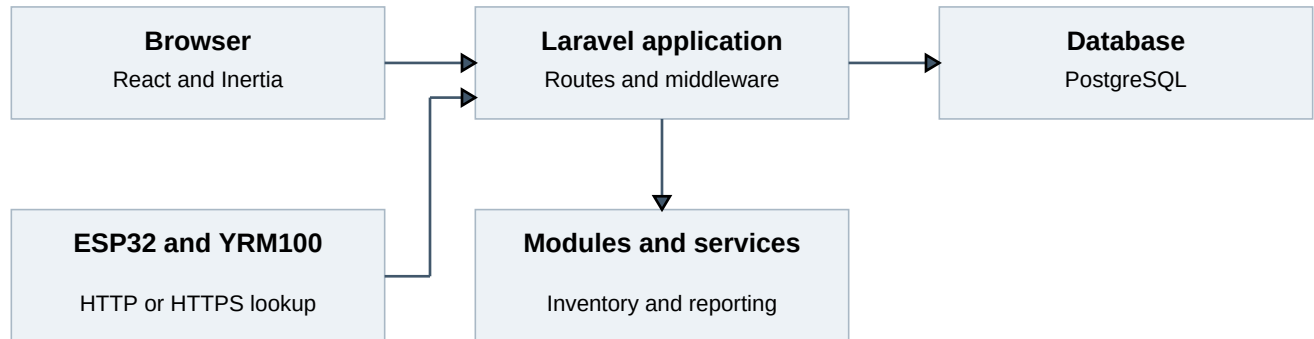
System administrators maintain accounts, permissions, settings, and signatories. Property or inventory staff record stock and prepare reports. Supervisors and reviewers consume summaries and trace records according to the approved permission model. The actual role matrix should be agreed with the project stakeholders.

Scope boundary

RFID currently identifies an item master through a tag association; a lookup does not itself receive or issue stock. Do not present the implementation as individual-unit asset tracking, autonomous stock counting, a full procurement lifecycle, or a formally certified accounting system without additional evidence.

System Architecture and Data Design

NEMIX is a modular monolith: one Laravel application organizes related capabilities into modules and shares a relational database. Inertia connects server routes and React pages. Docker Compose configures PostgreSQL; automated tests use an isolated SQLite memory database. [S1-S3]



Layer	Technology or responsibility
Presentation	React 18, TypeScript, Tailwind CSS, Inertia; Vite builds browser assets
Application	Laravel 12; controllers, validation requests, middleware and services
Business modules	Inventory, Suppliers, AuditLogs, UserManagement
Persistence	Eloquent models; PostgreSQL in Compose; 55 migration files
Device integration	ESP32 WROOM-32 and YRM100; Wi-Fi HTTP/HTTPS GET lookup
Supporting operations	Mail notifications, queue worker, cached RFID live feed

Core relationships

An item has receiving batches. A receiving links an item to a supplier; its batch preserves the supplier stock number and cost. An issuance has item lines, and each line can have multiple batch allocations. Allocations record which stock contributed to the issued quantity. Users attribute actions through issuances, receiving records, and audit trails. [S2-S3]

Design contribution

The distinction between an item master and a receiving batch allows the same item to arrive at different costs and from different suppliers. The recorded batch allocations make it possible to explain how an issuance value was obtained, rather than relying only on the item's latest unit cost.

Repository inventory: 205 PHP files across app, Modules, routes and database; 202 TSX files under resources; 42 test files. These counts indicate implementation size, not quality or test coverage.

Implemented Workflows and Feature Evidence

Capability	Evidence and qualification
Accounts and staff	Login, verification, password reset, staff invitations and OTP password changes have tests. Live email delivery was not tested.
Inventory receiving	Records quantities, costs, suppliers and batches; updates and voiding adjust stock.
Inventory issuance	Supports multiple lines, FIFO allocation, stock checks and reversals. Concurrent PostgreSQL execution remains untested.
Supplier management	Supplier records and valuation metrics are implemented and covered by dedicated tests.
Reporting	RSMI, RPCI, Stock Card and Memorandum Receipt snapshots have passing persistence tests; one older RSMI assertion fails.
Audit trails	Login and transaction histories include grouping and metadata; they are not proof of tamper-proof storage.
RFID	Tag assignment, duplicate rejection, lookup and live-feed cache exist. Hardware performance remains unmeasured.
Administration	Settings, mode controls, signatories and settings export exist. Settings export is not a full database backup.

Receiving to reporting flow

Select an existing item and supplier, record received quantity and price, and preserve the receiving batch. Issue an authorized quantity to a recipient or department. FIFO allocations determine the issued value and remaining batch balances. Review the resulting stock, supplier valuation, audit history, and report snapshot.

Illustrative FIFO example

For a clean demonstration item, receive 10 units at PHP 20 and then 5 units at PHP 25. Issuing 12 units should consume 10 from the first batch and 2 from the second: PHP 250 issued value and 3 units remaining at PHP 25, worth PHP 75. This is a rehearsal example, not an observed transaction from institutional data.

Reporting vocabulary

RSMI means Report of Supplies and Materials Issued; RPCI means Report on the Physical Count of Inventories; RIS means Requisition and Issue Slip. Official report stock numbers use supplier stock numbers, while internal SKUs identify item masters in the application. Form acceptance must be validated by the responsible office.

Verification Results and Interpretation

Audit method

The audit reviewed routes, middleware, service logic, data models, migrations, deployment files, firmware, and existing tests at revision d853b87eda0774d665f8b1718613749aac7cb99f. The working tree was clean before report generation. Application code was not changed. Test runs explicitly used SQLite :memory:, array mail/cache/session and synchronous queues; no production migration or device flashing was performed.

Check	Result and limits
Existing PHPUnit suite	273 tests; 272 passed; 1 failure; 1,682 assertions. SQLite only.
Batch architecture tests	25 tests passed, covering FIFO, allocation splitting, valuation and historical stock numbers.
Stock movement tests	7 tests passed, including insufficient-stock and reversal behavior.
Snapshot persistence tests	6 tests passed across reports and generated snapshots.
TypeScript	tsc --noEmit --pretty false passed; this is static analysis, not browser testing.
Vite production build	Succeeded in an audit output folder; public/build was not replaced.
Audit characterization	3 tests / 5 assertions confirmed the access behaviors described below.

The failing RSMI test

Isolec25010DefectFixesTest fails at line 234 after locating a report row by internal SKU. The current resolver explicitly avoids SKU fallback, and the newer stock-number standardization tests pass. The evidence points to a test/requirement mismatch rather than proven incorrect quantity totals. Confirm the agreed stock-number rule, give the fixture proper supplier stock numbers and allocations, and retain the quantity and cost assertions. Do not simply remove the test. [S10]

Meaning of the additional checks

The audit checks intentionally assert the observed permissive behavior. Their passing result confirms a weakness; it does not mean access control passed security testing. They are stored separately under docs/capstone-2/evidence and are excluded from the existing 273-test figure.

Evidence limits

No browser interaction, load test, restore rehearsal, physical scanner trial, user survey, dependency-advisory scan, or full penetration test was completed. No response-time improvement, usability score, ISO certification, or institutional compliance outcome is asserted.

High Priority Security Findings

F01 RFID authentication bypass

Severity: High. AuthenticateRfidHardware allows requests without a bearer token when the User-Agent contains ESP32, or when the configured token is empty. A client can choose this header. The existing RFID test and independent audit checks confirm acceptance. The shared middleware protects lookup and live feed as well as status, so the weakness can expose tagged item details and allow injected lookup events in the shared feed. [S6]

Recommendation: require a valid provisioned device credential, fail closed when configuration is missing, and check permissions for browser sessions. Remove the header exception. Acceptance: anonymous, spoofed-header and missing-configuration requests are denied; valid devices and authorized users succeed.

F02 Read permissions are not consistently enforced

Severity: High. AuthorizeAction skips GET and HEAD. Independent checks confirmed that a verified active user with no permissions receives HTTP 200 from /inventories and /rfid-scanner. The RFID view guard also combines a denied permission with membership in the same permission set, preventing the intended denial in the ordinary no-permission case. Hiding navigation does not restrict direct URL access. [S7]

Recommendation: enforce explicit view permissions on protected read routes and simplify the RFID authorization condition. Acceptance: a role matrix verifies both direct reads and writes, including roleless users and inactive sessions.

F03 Device configuration contains a tracked Wi Fi password

Severity: High if the configured password is active. git ls-files confirms that firmware/rfid-scanner/include/config.h is tracked, and the file contains a non-placeholder Wi-Fi password. The device bearer token is empty. This audit does not test whether the password is still valid; its value is intentionally omitted. [S8]

Recommendation: rotate any active exposed credential, provision local device settings outside version control, and commit a placeholder template. Coordinate any repository-history cleanup separately. Acceptance: tracked configuration and build artifacts contain no operational secrets.

F04 Firmware does not verify the HTTPS server

Severity: High in an untrusted network. api_client.cpp calls setInsecure() for HTTPS and also permits HTTP URLs. An encrypted connection without certificate validation does not authenticate the server. This is a code finding, not a demonstrated network interception. [S9]

Recommendation: use trusted CA validation, require HTTPS in deployment configuration, and test rejection of invalid certificates. Ensure reliable device time and an update process for trust material.

Additional Findings and Operational Gaps

F05 Predictable seeded accounts

Severity: Medium; potentially High if deployed unchanged. DatabaseSeeder creates administrator and staff accounts with fixed passwords and no production guard. Do not assume those accounts exist in deployment. Restrict demo seeds to development and use a one-time administrator provisioning flow. Verify that production cannot create usable default accounts. [S11]

F06 RSMI test and current stock number contract disagree

Severity: Medium. One test expects internal SKUs where current official-form logic requires supplier stock numbers. Agree the requirement, update the fixture, and rerun all tests. Expected closure is zero suite failures while retaining multi-item, quantity, and value checks. See the verification section for the exact failure. [S10]

F07 Settings export is not full recovery

Severity: Medium. exportBackup emits system configuration and settings only. It excludes inventory transactions, users, suppliers, audit trails and report history. This is useful configuration export but cannot recover the full application. Define a database and file backup procedure, retention and access rules, and a tested restore process. Measure recovery time and recoverable data loss in rehearsal. [S12]

F08 Reports load full history before filtering

Severity: Medium as a scalability risk. ComplianceReportController loads complete item and transaction collections; getRsmiRecords retrieves all issuances before filtering by period in PHP. RFID index also loads the full item catalog alongside a paginated table. Memory and response size may grow with dataset size; no load-related failure was measured. Use database-side filters, bounded selectors and paginated or streamed exports, then measure with representative data. [S13]

F09 Setup and architecture documentation are stale

Severity: Low. README uses laravel.test and port 8080, while current Compose defines app and port 80. The architecture blueprint mixes PostgreSQL prose with a MySQL diagram and predates batch services. Firmware README describes a future POST API, while the current implementation uses GET lookup. Update the onboarding steps, component/data diagrams, and device contract. [S14]

Additional review targets

Review trusted-proxy boundaries, sensitive-field redaction in exception logging, and client-supplied report snapshot integrity before production. These areas were observed during source review but were not fully exercised; they are follow-up targets, not confirmed exploit findings.

Quality Assessment and Completion Plan

Quality area	Assessment
Functional suitability	Strong automated evidence for stock workflows; resolve the RSMI assertion mismatch.
Reliability	Transactions, locks and reversal tests exist; PostgreSQL concurrency and restore tests remain.
Security	Implemented controls coexist with confirmed RFID and read-permission weaknesses.
Performance	Static build succeeds; full-history loading requires scale testing.
Usability and accessibility	UI implementation exists; no observed user study or accessibility audit.
Maintainability	Modules and inventory services help organization; documentation and test contracts need alignment.
Deployment and hardware	Compose and device source exist; actual deployment and scanner acceptance remain separate.

This is a qualitative engineering assessment. It is not an ISO/IEC 25010 certification or a weighted evaluation score. A test class name containing the standard does not establish conformance.

Before the defense

- Resolve the failing RSMI test against the approved requirement, then preserve the new test log.
- Close F01-F04 and rehearse using a disposable dataset with provisioned device credentials.
- Prepare one complete receiving-to-report example and one rejected over-issuance example.
- Check printed report pages, signatories and stock numbers against the office-approved forms.

Before a pilot deployment

- Test direct read and write access for every role, including negative cases.
- Run integration and competing-issuance tests on PostgreSQL; verify that stock never overshoots.
- Rehearse database/file restoration and verify row counts and report totals.
- Record scanner trials and user acceptance tasks with agreed success criteria.

Suggested measurements

Record scan trials, successful reads, false or duplicate reads, tag distance, orientation and lookup latency. For users, record task completion, errors, time and feedback before and after using NEMIX. For scale testing, record dataset size, concurrent users, median and 95th-percentile latency, and failures. Set acceptance thresholds with the stakeholders before testing; do not report proposed targets as achieved results.

Presentation Outline for the Defense

Suggested duration: 12-15 minutes, followed by panel questions. The following is a content outline for slides, not an automatically generated slide deck.

Slide	Message and supporting material
1 Project introduction	NEMIX title, team details and institution approved by the presenters. State the inventory scope.
2 Problem and users	Explain stock visibility, traceability and report preparation needs. Add actual stakeholder evidence.
3 Objectives and scope	Map receiving, issuance, suppliers, reporting and RFID to the objectives.
4 Architecture	Show browser, Laravel modules, database and device lookup connection. Explain a modular monolith.
5 Data design	Explain item master, batches, issuance lines and allocations with the FIFO example.
6 Inventory demonstration	Receive, issue, check remaining balance and reject an over-issuance.
7 Reports and audit trails	Show supplier stock number, signatories, snapshot, and actor/time/action history.
8 RFID demonstration	Assign a tag and look it up. Explain that lookup identifies an item and does not change stock.
9 Verification	Present 272/273 tests, 1,682 assertions, TypeScript pass and Vite pass; disclose the test mismatch.
10 Audit findings	Show the priority fixes, particularly device authentication and direct-route permissions.
11 Limits and next steps	State hardware, user acceptance, PostgreSQL concurrency and recovery validation needs.
12 Conclusion	Summarize the implemented contribution and the evidence required before production use.

Suggested opening

"NEMIX brings receiving, issuance, supplier records and inventory reporting into one application. Its central design separates an item from its receiving batches, allowing us to preserve different costs and trace stock as it is issued. We will demonstrate that workflow and explain both our verified results and remaining work."

Suggested closing

"The project implements the main inventory workflows and has broad automated test evidence. Our audit also identified specific access-control and verification gaps. The next milestone is to close those findings and complete supervised user, hardware and recovery testing before production deployment."

Demonstration Script and Evidence Checklist

Prepare the demonstration

Use a disposable demonstration database, approved sample names, two receiving batches, an administrator, a restricted staff account and a test RFID tag. Confirm the active mode, signatories and date filters. Keep operational passwords, tokens, real personal data and network configuration off screen.

Step	Action and expected checkpoint
1 Sign in	Open the dashboard and explain the authenticated role.
2 Receive stock	Create or select a clean item; receive 10 at PHP 20 and 5 at PHP 25. Expected stock: 15.
3 Issue stock	Issue 12 units. Explain FIFO: issued cost PHP 250; balance 3 units / PHP 75.
4 Reject excess	Try issuing 4 more with strict stock enforcement. Expect rejection and unchanged balance.
5 Review history	Show the receiving and issuance references, actor and timestamp.
6 Generate a report	Use the correct period and supplier stock number; review values, signatories and print preview.
7 Identify via RFID	Scan a known tag, then an unknown tag. Check the match result and explain that stock is unchanged.
8 Show limitations	Present test evidence and the remediation list. Do not demonstrate unresolved bypasses against a public server.

Capture for the presentation

- Dashboard and inventory balances before and after issuance.
- Receiving batches and allocation evidence for the numerical example.
- Rejected insufficient-stock request and a matching audit entry where applicable.
- Rendered report showing stock number, totals and signatories.
- Scanner photo or recorded local trial and its observed lookup result.
- Test summary and a dated issue-closure checklist.

Fallback if the scanner is unavailable

Use an explicitly labeled recorded hardware trial if one exists. Otherwise show the firmware architecture and a software-only lookup demonstration, clearly identifying it as API behavior rather than proof of physical scanning. Never replace absent field results with estimated accuracy or range.

Likely Panel Questions and Suggested Answers

What is the main technical contribution

"We separated the item master from receiving batches and preserve issuance allocations. That allows one item to have multiple supplier deliveries and costs while supporting traceable stock and value calculations."

How do you prevent negative stock

"The issuance service checks available stock within a database transaction and locks relevant records. Automated tests cover over-issuance rejection and reversal. We still need competing-request tests on PostgreSQL to validate concurrency under the deployment database."

Why are you using FIFO

"The implemented policy consumes the oldest eligible batches first and records their contribution to the issuance. We will confirm that this policy matches the office's approved inventory procedure."

Does RFID automatically count inventory

"The implemented path identifies an item associated with a tag and updates the live lookup feed. It does not automatically count all units or post a receiving or issuance transaction."

Is the system secure

"It includes authentication, roles, validation and audit trails, but our audit identified specific device-authentication and read-permission gaps. We are treating their closure and negative access tests as requirements before external deployment."

Why did one test fail

"An older RSMI test searches by internal SKU, while current reporting intentionally uses supplier stock numbers. We need to align the fixture with the approved rule and rerun all quantity and amount checks. We are reporting the failure rather than calling the suite fully passed."

How did you evaluate usability and accuracy

"This audit provides automated and source-level evidence. User acceptance, usability, physical RFID accuracy and actual processing-time improvements require observed trials. We will report the participants, tasks, sample sizes and results when those trials are complete."

Can the backup restore the whole system

"The current application export saves settings. Full recovery requires database and file backups and a successful restoration test. Those are separate acceptance requirements."

Are all forms officially compliant

"The project generates the named forms and tests several data rules. Official acceptance must come from the responsible office after reviewing layout, fields, totals and signatories; the code alone does not establish certification."

Evidence Register and Reproduction Notes

Source references are repository-relative paths at the audited revision. Line numbers refer to that revision. Raw command results and the independent audit checks are retained under docs/capstone-2/evidence. No secret values are included in this report.

S1 composer.json; package.json; modules_statuses.json; docker-compose.yml;
app/Http/Controllers/Admin/SystemSettingController.php, testEmail. Stack, module registration, database configuration and institutional branding.

S2 Modules/Inventory/app/Services/InventoryReceivingService.php:103; InventoryIssuanceService.php:35;
InventoryCostingService.php:27. Receiving transactions, issuance and FIFO allocation. Paths after the first share the same Services directory.

S3 database/migrations/2026_09_12_000002_create_inventory_batches_and_allocations_tables.php;
Modules/Inventory/app/Models. Batch and allocation schema and relationships.

S4 tests/Feature/Inventory/ItemMasterBatchArchitectureTest.php; tests/Feature/Inventory/StockMovementTest.php. Stock, costing, historical value and reversal scenarios.

S5 tests/Feature/ComplianceReportSnapshotPersistenceTest.php; app/Http/Controllers/Compliance/ComplianceReportController.php. Persisted report datasets and snapshot behavior.

S6 app/Http/Middleware/AuthenticateRfidHardware.php:38; routes/web.php; tests/Feature/Inventory/RfidScannerTest.php. Header/configuration bypass and affected hardware endpoints.

S7 app/Http/Middleware/AuthorizeAction.php:18; app/Http/Controllers/Inventory/RfidScannerController.php:23;
docs/capstone-2/evidence/AuditVerificationTest.php. Read-route bypass and independent reproduction.

S8 firmware/rfid-scanner/include/config.h:9 and :22; git ls-files result. Tracked Wi-Fi configuration and empty device token. Values redacted.

S9 firmware/rfid-scanner/src/api_client.cpp:95-107. TLS validation disabled and HTTP fallback supported.

S10 tests/Feature/Isolec25010DefectFixesTest.php:233; app/Services/Compliance/ComplianceReportDataService.php:305;
tests/Feature/Compliance/ComplianceFormsStockNumberStandardizationTest.php. Failing SKU expectation and newer stock-number contract.

S11 database/seeders/DatabaseSeeder.php. Fixed credential account provisioning without production guard.

S12 app/Http/Controllers/Admin/SystemSettingController.php:212. Export contains configuration and settings only.

S13 app/Http/Controllers/Compliance/ComplianceReportController.php:18;
app/Services/Compliance/ComplianceReportDataService.php:367; app/Http/Controllers/Inventory/RfidScannerController.php:44. Unbounded collection loading and filtering after retrieval.

S14 README.md; SYSTEM_ARCHITECTURE_BLUEPRINT.md; docker-compose.yml; firmware/rfid-scanner/README.md. Documented configuration differs from current implementation.

Reproducibility and Final Readiness Decision

Commands executed

Backend: `php vendor/phpunit/phpunit/phpunit --log-junit docs/capstone-2/evidence/phpunit.xml --do-not-cache-result`

Frontend static check: `node node_modules/typescript/bin/tsc --noEmit --pretty false`

Frontend build: `node node_modules/vite/bin/vite.js build --outDir docs/capstone-2/evidence/build`

Independent checks: `php vendor/phpunit/phpunit/phpunit docs/capstone-2/evidence/AuditVerificationTest.php --do-not-cache-result`

Firmware: `arduino-cli compile --fqbn esp32:esp32:esp32 --build-path docs/capstone-2/evidence/firmware-build`
`firmware/rfid-scanner`

Execution notes

The backend and independent checks used `APP_ENV=testing`, `DB_CONNECTION=sqlite`, `DB_DATABASE=:memory:`, `MAIL_MAILER=array`, `CACHE_STORE=array`, `SESSION_DRIVER=array` and `QUEUE_CONNECTION=sync`. This isolates database writes from the configured application database. The frontend build and ESP32 compile initially encountered sandbox process restrictions and were retried with approved escalation.

Firmware compilation passed for `esp32:esp32:esp32` using installed ESP32 core 3.3.11. This proves the sketch compiles in this environment; it does not prove flashing, radio read performance, battery behavior, network reliability or physical integration.

Audit artifacts

File under evidence	Purpose
<code>phpunit.txt</code> and <code>phpunit.xml</code>	Original suite console output and machine-readable results
<code>typescript.txt</code>	Static-check result and explicit exit code
<code>build.txt</code>	Frontend production build output
<code>audit-verification.txt</code>	Three reproduced permissive access behaviors
<code>AuditVerificationTest.php</code>	Independent characterization checks
<code>firmware.txt</code>	ESP32 compilation output

Final readiness decision

Capstone presentation: suitable as an implemented prototype, subject to rehearsal and honest disclosure of the failing assertion and open findings. Pilot or production deployment: withhold readiness claims until device authentication, direct-route permissions, secret handling, test alignment and recovery validation are closed. Physical RFID and stakeholder acceptance results must be recorded separately.

The next evidence update should retain the reviewed revision, test results, issue closures and acceptance records together. This makes progress defensible without relying on unchecked completion lists or unsupported percentages.